

对象的样式 (Styles)

什么是样式

样式是什么意思？

样式 (样式的含义)

播报

编辑

讨论

上传视频

样式：用名称保存下来的对修饰对象进行修饰所使用的一组修饰参数。

修饰对象：字符、段落、链接段落和字符（用于做超链接的段落和字符）、表格和列表。

在lvgl中styles 用于设置对象的外观：

- 样式是一个 lv_style_t 变量，它可以保存边框宽度、文本颜色等属性。
- 将样式(变量)分配给对象就可以改变其外观。在赋值过程中，可以指定目标部分和目标状态。
- 一个样式可以给多个对象使用（正常样式）。
- 样式可以级联，也就是可以将多个样式分配给一个对象。所以，我们不用将所有属性都在一个样式中指定，可以通过多个样式组合的形式指定。LVGL 会优先使用我们定义的样式，如果没有就会使用默认值。
- 后来添加的样式具有更高的优先级。也就是说如果在两种样式中指定了同一个属性，则将使用最后添加的样式。
- 如果对象中未指定某些属性（例如文本颜色），就会从父级继承。
- 上面说的是“正常”样式，对象还有本地样式，它比“正常”样式具有更高的优先级。
- 可以定义有过渡效果的样式。
- 默认有一个样式主题，我们也可以自己定义样式主题，作为默认的样式主题使用。

样式的使用

初始化样式

样式存储在 lv_style_t 变量中。样式变量应该是 静态、全局或动态分配 的。也就是它们不能是函数中的局部变量，因为当函数结束时它们会被销毁。样式初始化示例：

```
static lv_style_t style_obj;
lv_style_init(&style_obj);
```

设置样式属性

当我们初始化好一个样式之后就可以设置它的样式属性了，接口函数是这样的格式：

```
lv_style_set_<property_name>(&style, <value>);
```

示例：

```
lv_style_set_bg_color(&style_obj, lv_color_hex(0x000000)); // 设置背景色
lv_style_set_bg_opa(&style_obj, LV_OPA_50);                // 设置背景透明度
lv_style_set_....
```

添加(应用)样式到对象

当我们初始化并且设置好一个样式之后就可以将它添加到对象上面了，接口函数只有一个：

```
lv_obj_add_style(obj, &style, <selector>)
```

参数“obj”就是要添加到的对象，“style”是指向样式变量的指针，是应添加样式的部分和状态的 OR-ed 值 (不能是互斥，否则就是清除标志，没法合并)。示例：

```
lv_obj_add_style(obj, &style_obj, 0);                // 默认(常用)
lv_obj_add_style(obj, &style_obj, LV_STATE_PRESSED); // 在对象被按下时应用样式
```

获取样式属性

我们可以获取属性的最终值（考虑级联、继承、本地样式和转换），接口函数是这样的格式：

```
lv_obj_get_style_<property_name>(obj, <part>);
```

函数使用对象的当前状态，如果没有更好的候选对象，则返回默认值。例如：

```
lv_color_t color = lv_obj_get_style_bg_color(obj, LV_PART_MAIN);
```

删除样式

删除对象的所有样式：

```
lv_obj_remove_style_all(obj);
```

删除对象的特定样式：

```
lv_obj_remove_style(obj, &style_obj, selector);
```

只有当 selector 与 lv_obj_add_style 中使用的 selector 匹配时，此函数才会删除 style

如果 style 是空，那么会根据给出的 selector 检查并删除所有匹配的样式

如果 selector 是 LV_STATE_ANY 或 LV_PART_ANY 就会删除具有任何状态或部分的样式。下面这个效果和 lv_obj_remove_style_all 的效果是一样的：

```
lv_obj_remove_style(obj, NULL, LV_STATE_ANY | LV_PART_ANY );
```

查看样式属性

所有的可用的样式属性我们可以在文档或者代码中获取得到。

- 文档位置：
 - 英文原版: <https://docs.lvgl.io/9.1/overview/style-props.html>
 - 中文翻译: <http://lvgl.100ask.net/9.1/overview/style-props.html>
- 代码位置：
 - 普通样式: lvgl/src/misc/lv_style_gen.h
 - 本地样式: lvgl/src/core/lv_obj_style_gen.h

文档位置和代码位置可能在后续的版本更新中会发生变化，这里的方法只是提供参考，不需要死记硬背函数接口名。

背景属性和我们前面学习的盒子模型关系很大，背景属性主要有一下这些：

- 背景(Background)
- 边界(Border)
- 轮廓(Outline)
- 阴影(Shadow)
- 填充(Padding)
- 宽度和高度变换
- X和Y变换

样式的状态和部分

状态(States)

对象可以处于以下状态的组合：

- LV_STATE_DEFAULT (0x0000) 正常，释放状态
- LV_STATE_CHECKED (0x0001) 切换或检查状态

- LV_STATE_FOCUSED (0x0002) 通过键盘或编码器聚焦或通过触摸板/鼠标点击
- LV_STATE_FOCUS_KEY (0x0004) 通过键盘或编码器聚焦，但不通过触摸板/鼠标聚焦
- LV_STATE_EDITED (0x0008) 由编码器编辑
- LV_STATE_HOVERED (0x0010) 鼠标悬停（现在不支持）
- LV_STATE_PRESSED (0x0020) 被按下
- LV_STATE_SCROLLED (0x0040) 正在滚动
- LV_STATE_DISABLED (0x0080) 禁用状态
- LV_STATE_USER_1 (0x1000) 自定义状态
- LV_STATE_USER_2 (0x2000) 自定义状态
- LV_STATE_USER_3 (0x4000) 自定义状态
- LV_STATE_USER_4 (0x8000) 自定义状态

这些可能会随着lvgl的更新而不断增加，同学们可以阅读[最新版本的文档](#)获取最新资料。

部分(Part)

对象可以有 部分(parts)，它们也可以有自己的样式。LVGL 中存在以下预定义部分：

- LV_PART_MAIN 类似矩形的背景
- LV_PART_SCROLLBAR 滚动条
- LV_PART_INDICATOR 指标，例如用于滑块、条、开关或复选框的勾选框
- LV_PART_KNOB 像手柄一样可以抓取调整值
- LV_PART_SELECTED 表示当前选择的选项或部分
- LV_PART_ITEMS 如果小部件具有多个相似元素（例如表格单元格）
- LV_PART_TICKS 刻度上的刻度，例如对于图表或仪表
- LV_PART_CURSOR 标记一个特定的地方，例如文本区域或图表的光标
- LV_PART_CUSTOM_FIRST 可以从这里添加自定义部件。

这些可能会随着lvgl的更新而不断增加，同学们可以阅读最新版本的文档获取最新资料。

案例

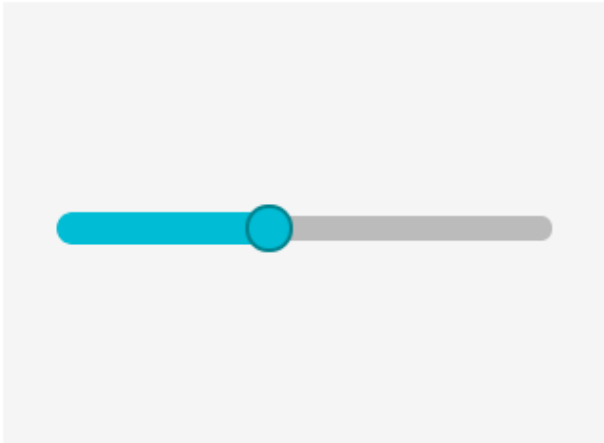
例如一个 [滑杆\(Slider\)](#) 包含三个部分：

- 背景

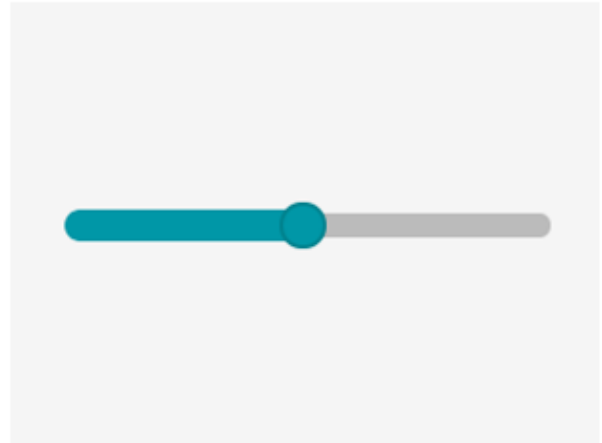
- 指标
- 旋钮

这也就是说滑块的这三个部分都可以有自己的样式。

示例体验: <http://lvgl.100ask.net/9.1/widgets/slider.html#slider-with-custom-style>



默认状态时



调整滑杆时

本地样式

- 除了“普通”样式外,对象还可以存储 本地样式(私有样式)。
- 本地样式与普通样式类似,但是它不能在其他对象之间共享。如果使用本地样式,将自动分配局部样式,并在删除对象时释放。本地样式对于向对象添加本地自定义很有用。
- 本地样式的接口函数是这样的格式: `lv_obj_set_style_<property_name>(obj, ,);`

示例:

```
lv_obj_set_style_bg_color(obj, lv_color_hex(0xffffffff), 0); // 设置背景色
lv_obj_set_style_bg_opa(obj, LV_OPA_50, 0); // 设置背景透明度
lv_style_set_style....
```

- 删除本地样式的时候我们只删除某一个样式:

```
lv_obj_remove_local_style_prop(obj, LV_STYLE_..., selector);
```

LV_STYLE_...的取值请看: `lvgl/src/misc/lv_style.h` 中的 `lv_style_prop_t`

样式继承

- 某些属性(通常与文本相关)可以从父对象的样式继承。

- 只有没有在为对象设置样式属性时，才应用继承。在这种情况下，如果这个属性是可继承的，那么这个属性的值会在父类中检索，直到一个对象为该属性指定了一个值。父类将使用自己的状态来确定该值。因此，如果按下按钮，并且文本颜色来自此处，则将使用按下的文本颜色。

过渡特效

默认情况下，当一个对象改变状态（例如它被按下）时，新状态的新属性会立即设置。但是，通过转换，可以在状态更改时播放过渡效果。例如，按下按钮时，其背景颜色可以在 300 毫秒内动画显示为按下的颜色。

demo体验：

- <https://docs.lvgl.io/9.1/overview/style.html#transition>
- <http://lvgl.100ask.net/9.1/overview/style.html#transition>

这部分内容在后面的课程再展开讨论。

样式主题

主题是风格的集合。如果存在活动主题，LVGL将其应用于每个创建的部件(对象)。这将为UI提供一个默认外观，然后通过添加更多样式对其进行修改。

demo体验：

- <https://docs.lvgl.io/9.1/overview/style.html#extending-the-current-theme>
- <http://lvgl.100ask.net/9.1/overview/style.html#extending-the-current-theme>

这部分内容在后面的课程再展开讨论。

文档位置

- <https://lvgl.100ask.net/9.1/overview/style.html>